

Breaking the Speed Limit

Fast statistical models with Python 3.14, Numba, and JAX

Simulation creates a validation target; speed only counts after the statistic is preserved.

Wenxin Jiang · Jian Yin

Department of Biostatistics, City University of Hong Kong · PyCon US 2026



github.com/LucaJiang/
FastStatisticalModels4Python

1. Why simulation is the statistical test harness

Real biomedical data rarely tells us the true clustering, the exact null distribution, or whether a numerical shortcut changed the estimand. Simulation supplies an answer key that ordinary benchmark timing cannot provide.

Define target what should be recovered, calibrated, or estimated

Generate scenarios vary signal, dimension, sample size, repeats, and hard cases

Reference first write the readable implementation before optimizing

Validate equivalence, recovery, null calibration, tolerance

Scale only then measure bottlenecks and choose an engine

Governing rule

A faster implementation that changes the statistic is a different method, not a speedup.

2. What a statistician has to decide

Question	Decision before timing
Scientific target Ground truth	estimator, statistic, null model, stopping rule simulated labels, known effect, or nominal type-I rate
Acceptance	exact match, tolerance, calibration band, recovery threshold
Failure meaning	wrong implementation, weak method, or systems limit
Scale stress	which dimensions make the computation expensive

Reference

golden oracle

Scenario grid

property / fuzz / load tests

Gate

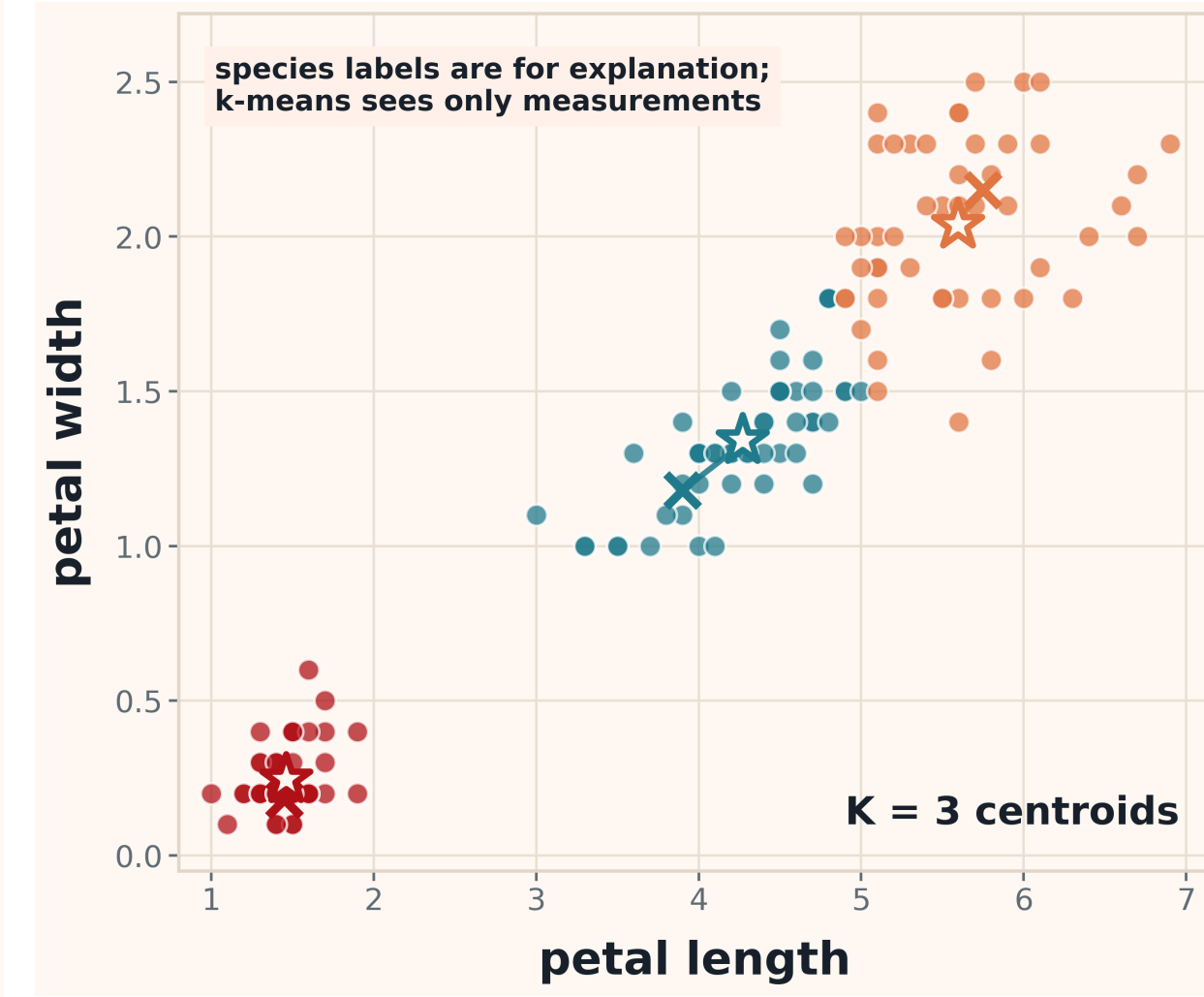
optimization is admitted only
after validation

3. Two workloads, two pressure shapes

These examples are not here to teach algorithms. They stand for two common ways simulations stress statistical code.

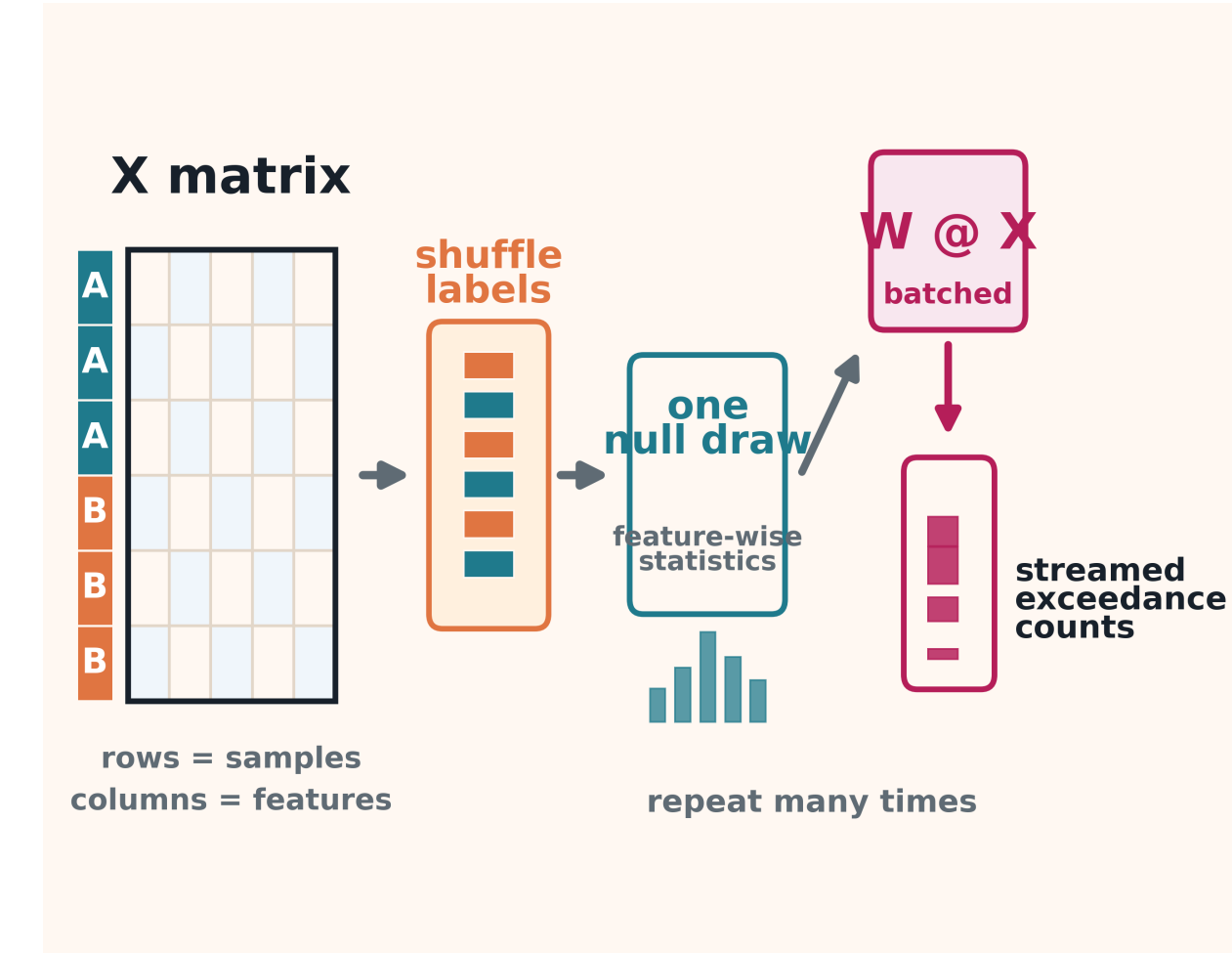
k-means

Iterative fitting: each update depends on the previous state.



permutation test

Resampling inference: repeated null draws across many features.



Validation contracts

k-means: same data, initialization, stopping rule; compare final inertia and recovery.

permutation: same permutation stream and p-value definition; check null calibration before runtime.

4. Evidence examples, scoped

Claim type	Committed evidence used in the talk
k-means equivalence	max relative inertia difference 3.1×10^{-14} ; tolerance 10^{-8}
permutation equivalence	max p-value difference 0.0; max statistic difference 9.4×10^{-16}
null behavior	MacBook null calibration mean $p \leq 0.05$ was 0.051
A100 boundary	first streamed-reduction win at $n = 5,000$, $p = 10,000$, $R = 5,000$, batch $R = 8,192$
A100 scope	largest slide-level measured speedup 8.54x at $p = 500,000$, $R = 5,000$

Timing semantics

Speedup = matched CPU matrix baseline / A100 streamed full end-to-end.
Compile excluded, transfer included, kernel-only excluded.

5. Choose the smallest tool that matches the bottleneck

Simulation signal	Conservative tool choice
result does not recover target	fix statistic / model before optimizing
hot scalar CPU loop	Numba
dense distance algebra	NumPy / BLAS
repeated shared-array work	threads / free-threaded context
large device-resident batch	JAX / A100
giant temporary array	rewrite or stream the computation
non-monotone parallel sweep	tune under resource constraints

trust first

compile loops

use algebra

batch for device

6. What to report with every speed claim

- validation target and acceptance tolerance;
- environment tier: local validation, server CPU, or A100;
- cold vs warm timing and whether compile is excluded;
- whether transfer, allocation, and collection are included;
- unavailable/OOM cells as unavailable, not as CPU wins;
- the smallest tool that preserves the statistical result.

Takeaway

1

Simulate known behavior

2

Preserve the statistic

3

Measure the bottleneck

4

Choose the smallest tool

Clear enough to trust. Fast enough to scale.

github.com/LucaJiang/FastStatisticalModels4Python